

Lab 6:

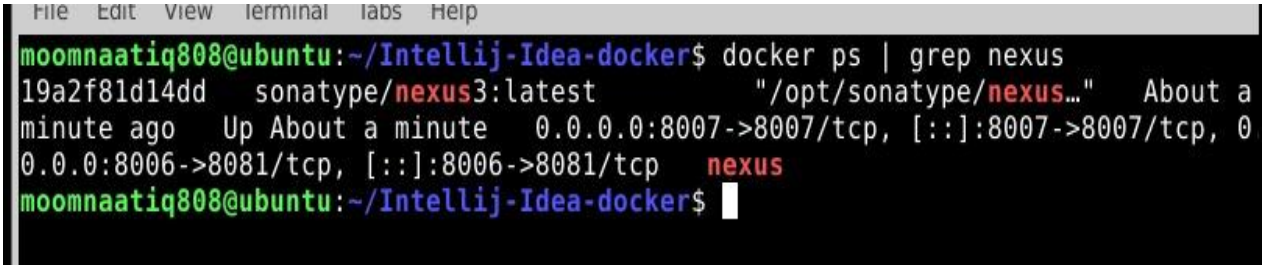
Set Up Nexus Repository and Configure User Access

Name: Moomna Atiq
GitHub Username: Moomnaatiq808
Roll No:23-BCS-065
Repository / Directory: DevOps / Labs/Lab06

Task 1: Run Nexus Repository and Complete the Initial Setup

nexus_container_running.png

The Nexus container was started with Docker on the Ubuntu server. The docker ps | grep nexus output shows it running with ports 8006 and 8007 published.

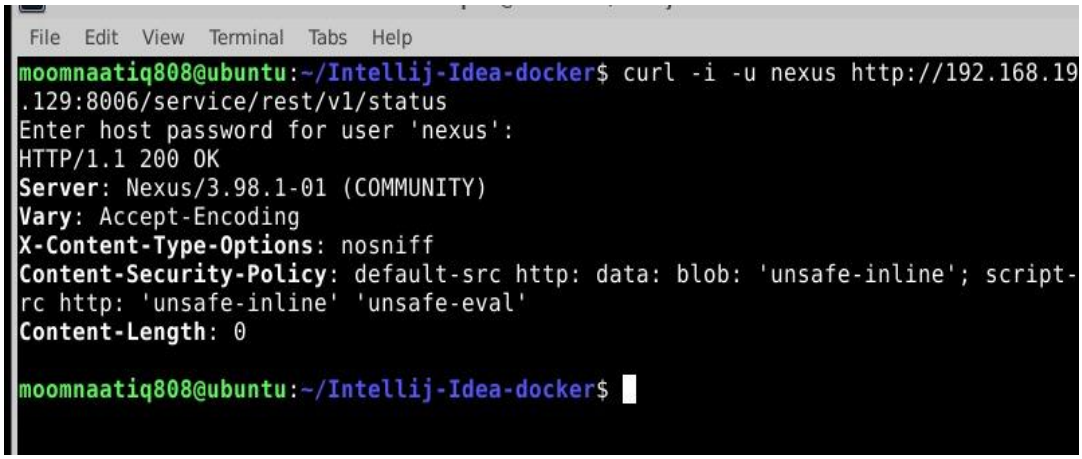


Summary: Nexus is running in Docker with persistent data, the initial administrator password has been replaced, and anonymous access is disabled.

Task 2: Create a Role and User and Verify Access

nexus_user_access.png

The nx-java role and the active nexus user were created. The curl command to the Status API returned HTTP/1.1 200 OK, which confirms the user can authenticate.



Summary: The local nexus user is active, receives Maven snapshots access through the nx-java role, and can authenticate to Nexus.

Task 3: Build and Publish the Gradle Application to Nexus

gradle_nexus_configuration.png

build.gradle was updated with the maven-publish plugin, the publishing block and the Nexus snapshot repository URL. settings.gradle sets rootProject.name = 'my-app'.



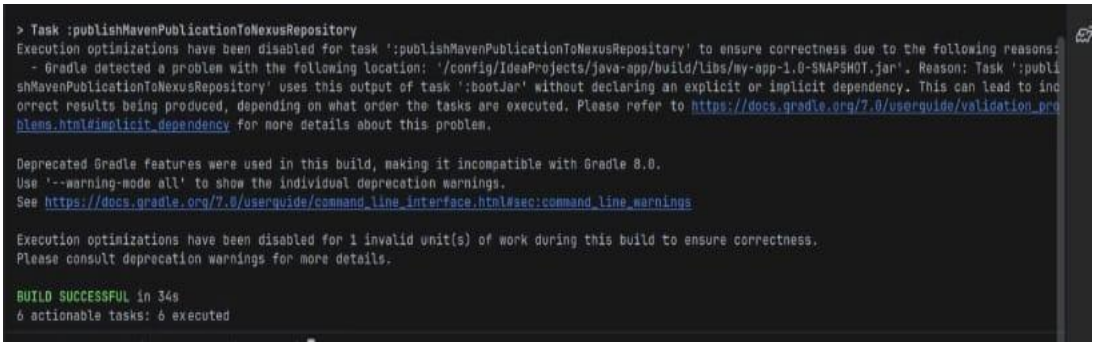
gradle_nexus_build.png

./gradlew clean build finished with BUILD SUCCESSFUL. The my-app-1.0-SNAPSHOT.jar file was generated in build/libs.



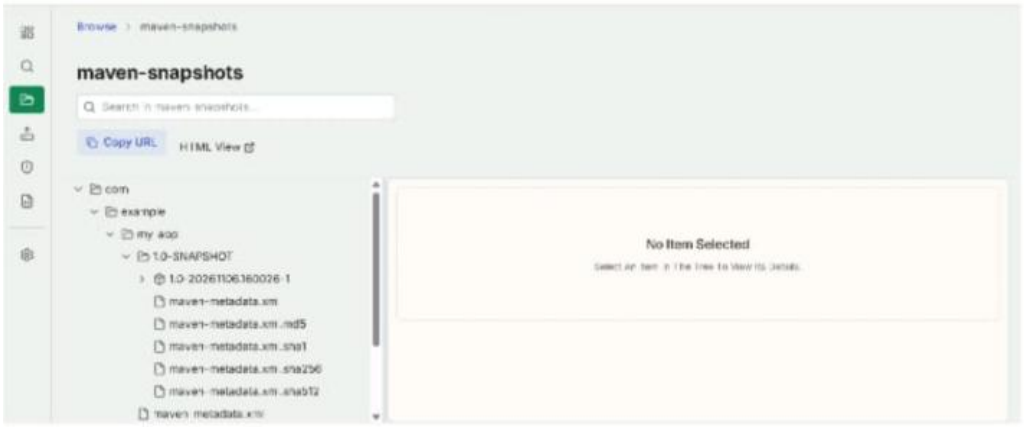
gradle_nexus_publish.png

./gradlew clean build publish ran the Nexus publish task and finished with BUILD SUCCESSFUL. The credentials were read from gradle.properties.



nexus_gradle_artifact.png

In Nexus, maven-snapshots shows com/example/my-app/1.0-SNAPSHOT with the published JAR and Maven metadata.

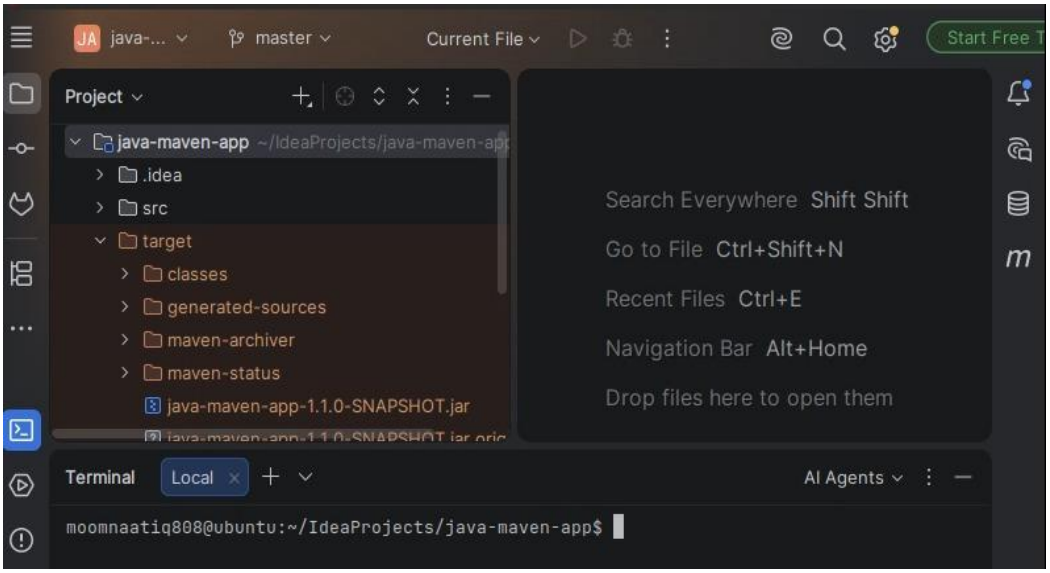


Summary: IntelliJ used JDK 16 to build the renamed my-app Gradle project. Gradle read the deployment credentials from the uncommitted project-level gradle.properties file and published the snapshot JAR and Maven metadata to Nexus.

Task 4: Publish the Maven Application to Nexus

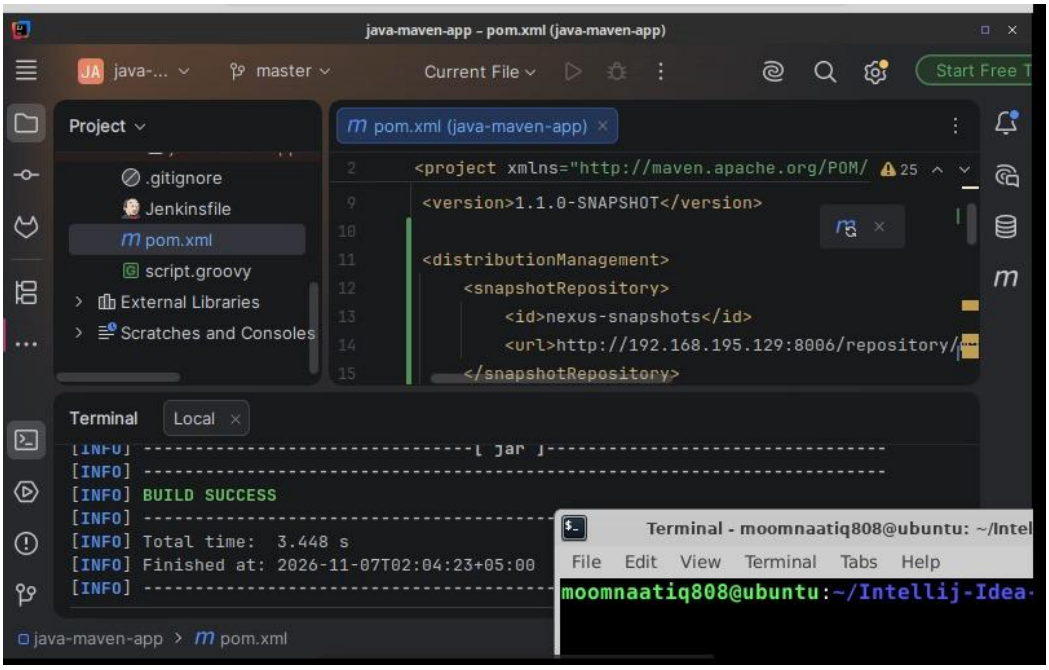
maven_nexus_project.png

The java-maven-app project was cloned/opened successfully in IntelliJ IDEA.



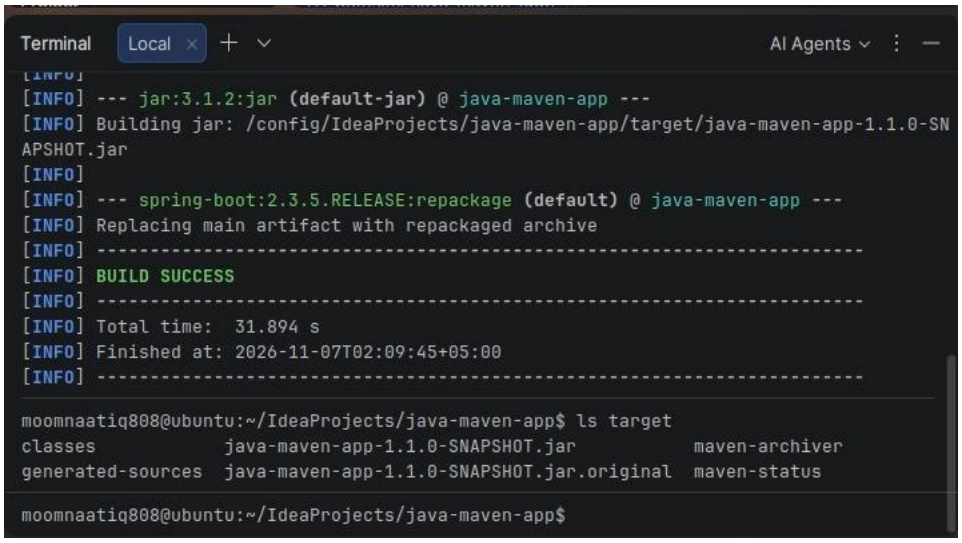
maven_distribution_management.png

The distributionManagement block with the nexus-snapshots snapshotRepository was added to pom.xml as a direct child of the project element.



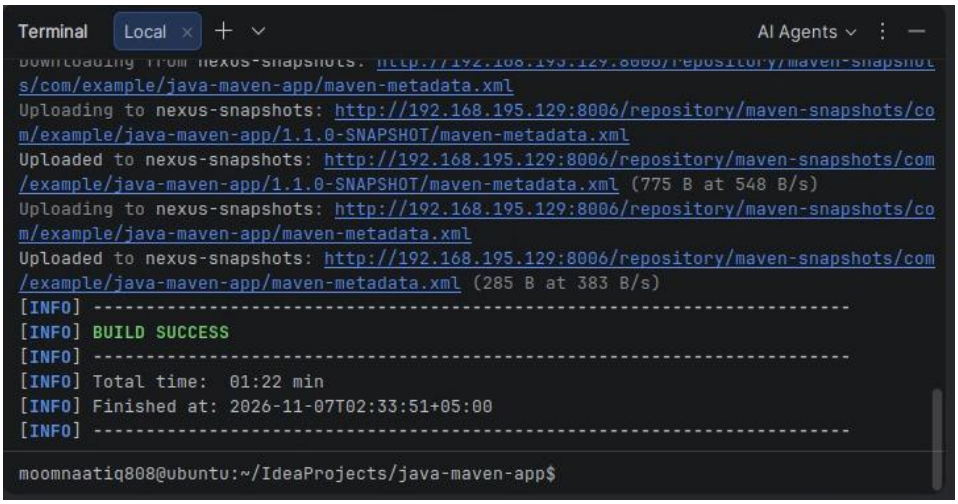
maven_package_nexus.png

mvn package completed with BUILD SUCCESS and created the snapshot JAR under the target directory.



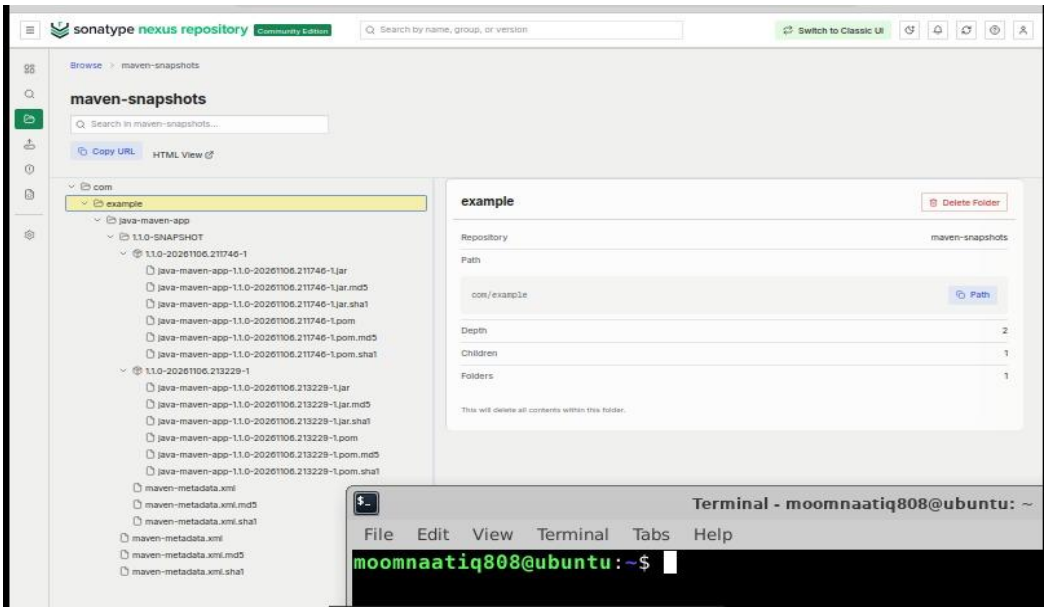
maven_deploy_nexus.png

mvn deploy uploaded the JAR, POM and metadata to the Nexus snapshot repository and finished with BUILD SUCCESS.



nexus_maven_artifact.png

In Nexus, maven-snapshots shows com/example/java-maven-app/1.1.0-SNAPSHOT with the JAR, POM and Maven metadata files.

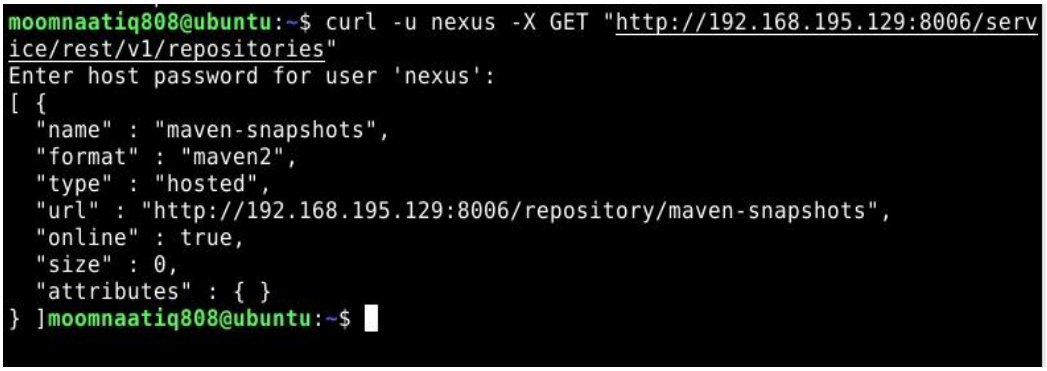


Summary: Maven read the deployment destination from pom.xml, matched it to the private credentials in ~/.m2/settings.xml, packaged the snapshot JAR and deployed the artifact and metadata to Nexus.

Task 5: Inspect Nexus Repositories and Components Using the REST API

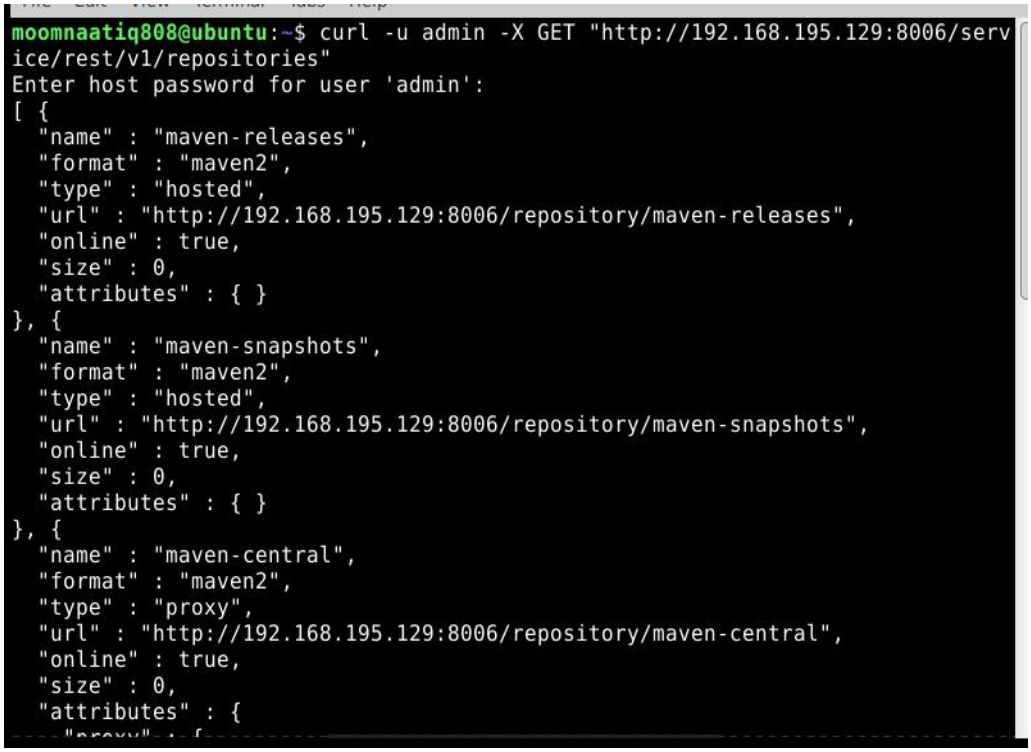
nexus_repositories_user.png

The repository list was requested through the REST API as the restricted nexus user. The response shows the repositories visible to this user.



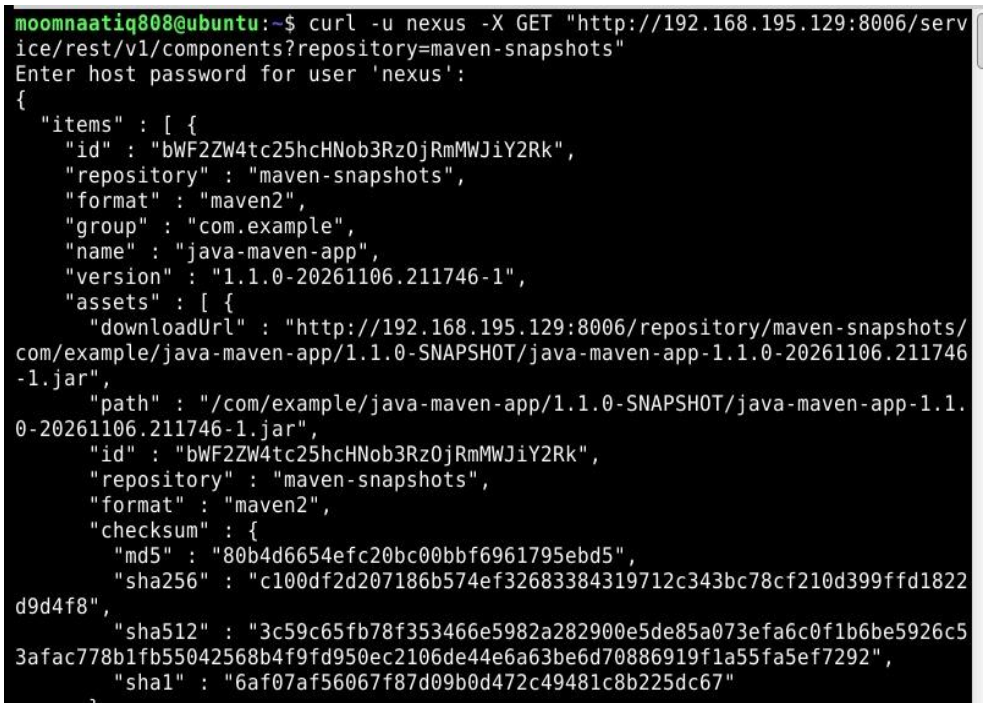
nexus_repositories_admin.png

The same repository list was requested as the admin user. The admin response can be compared with the nexus user's response.



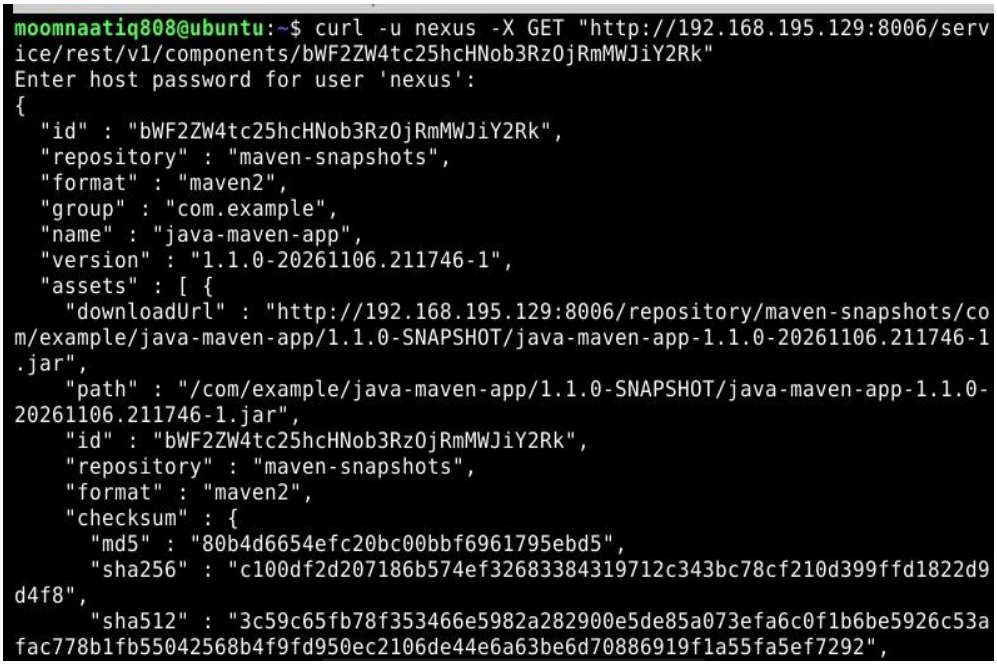
nexus_components_list.png

The components of maven-snapshots were listed as the nexus user. The response shows a component with its id, repository, name and version.



nexus_component_details.png

One component was requested by its exact ID. The response shows its repository, format, group, name, version and assets.

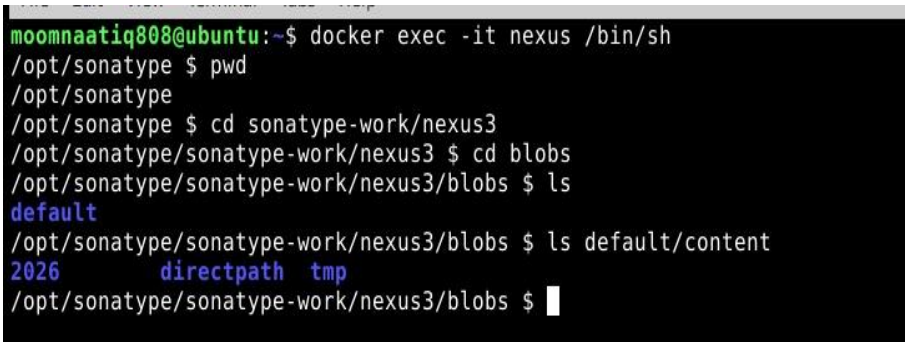


Summary: You queried the Nexus repositories with both the restricted nexus account and the admin account, listed the components in maven-snapshots, and retrieved one component by its REST API ID without exposing either password.

Task 6: Create a Nexus Blob Store and Docker Hosted Repository

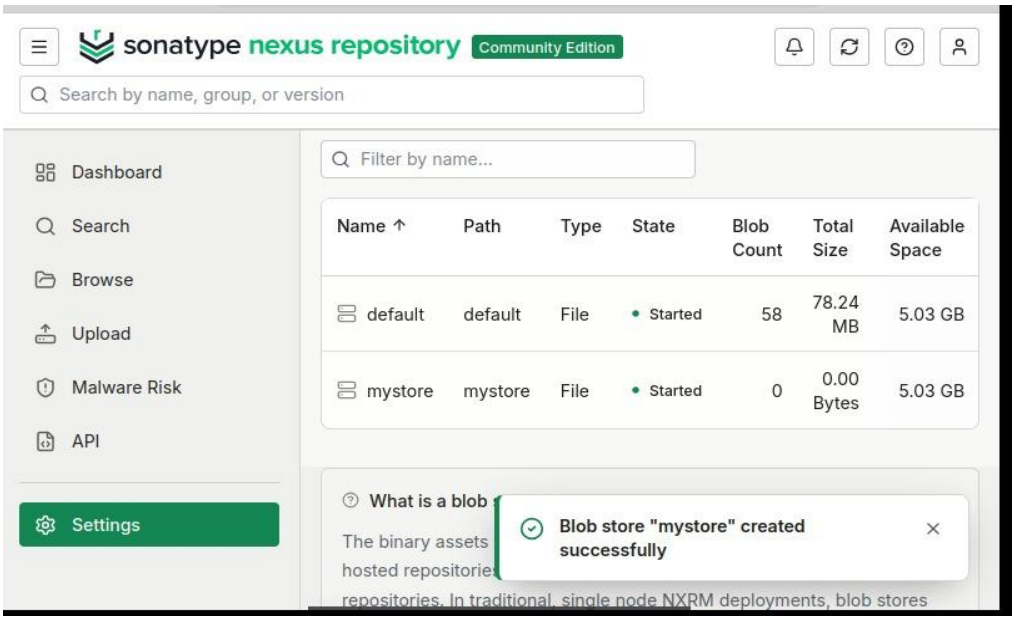
nexus_default_blob_content.png

A shell was opened in the Nexus container with docker exec to inspect the default blob store. The output shows the blobs path, the default directory and the content of default/content.



nexus_mystore_created.png

The mystore file blob store was created at /nexus-data/blobs/mystore. The Blob Stores page shows both default and mystore in the Started state.



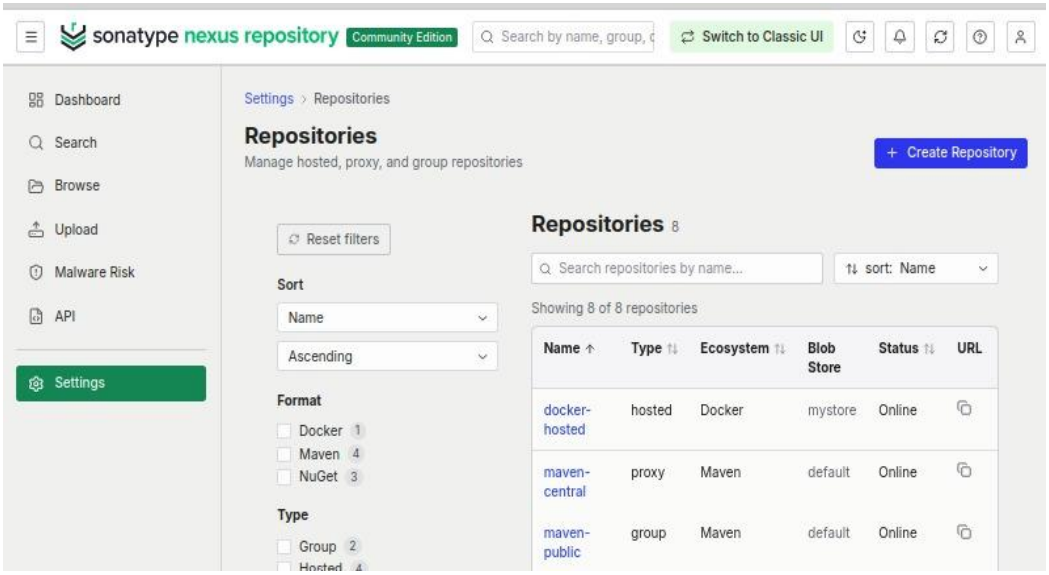
nexus_blob_directories.png

Inside the same container shell, the blobs path shows both the default and mystore directories. After exit, the Ubuntu prompt is shown again.

```
moomnaatiq808@ubuntu:~$ docker exec -it nexus /bin/sh
/opt/sonatype $ pwd
/opt/sonatype
/opt/sonatype $ cd sonatype-work/nexus3
/opt/sonatype/sonatype-work/nexus3 $ cd blobs
/opt/sonatype/sonatype-work/nexus3/blobs $ ls
default
/opt/sonatype/sonatype-work/nexus3/blobs $ ls default/content
2026        directpath  tmp
/opt/sonatype/sonatype-work/nexus3/blobs $ pwd
/opt/sonatype/sonatype-work/nexus3/blobs
/opt/sonatype/sonatype-work/nexus3/blobs $ ls
default  mystore
/opt/sonatype/sonatype-work/nexus3/blobs $ ls mystore
content             metadata.properties
/opt/sonatype/sonatype-work/nexus3/blobs $ exit
moomnaatiq808@ubuntu:~$
```

nexus_docker_hosted_repository.png

The docker-hosted repository was created as docker (hosted), Online, using the mystore blob store and HTTP connector port 8007.

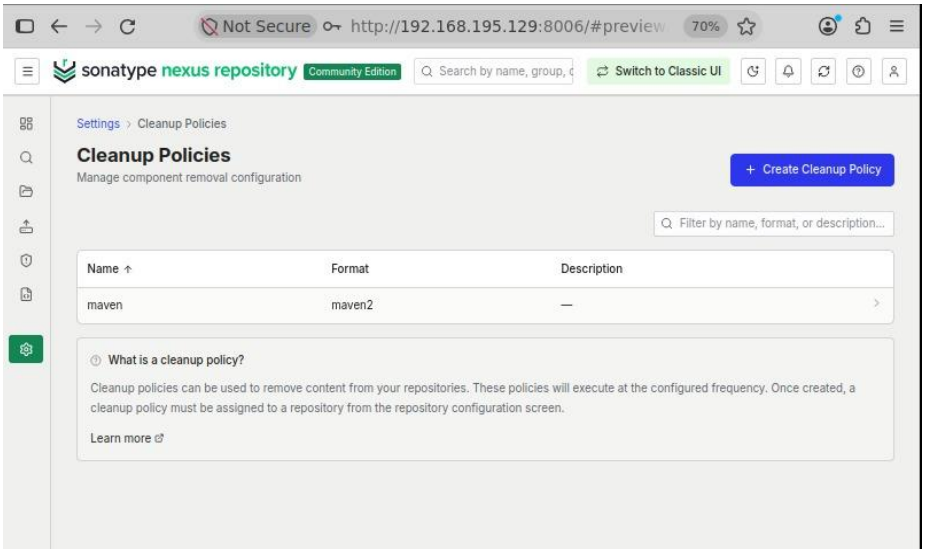


Summary: You inspected Nexus blob storage without modifying its files, created the mystore file blob store, verified its directory inside the container, and created an online Docker hosted repository that uses mystore through connector port 8007.

Task 7: Create and Run a Maven Snapshot Cleanup Policy

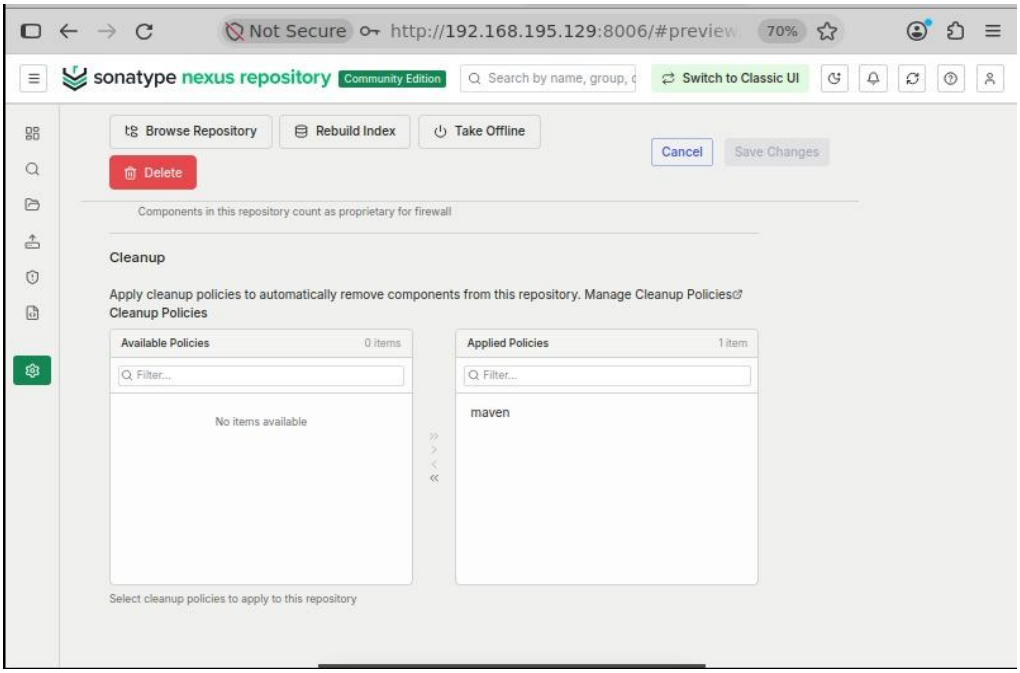
nexus_cleanup_policy_created.png

The maven cleanup policy was created with maven2 format, Pre-Releases/Snapshots release type and the .* asset name matcher. The preview of maven-snapshots lists the Gradle and Maven snapshot components.



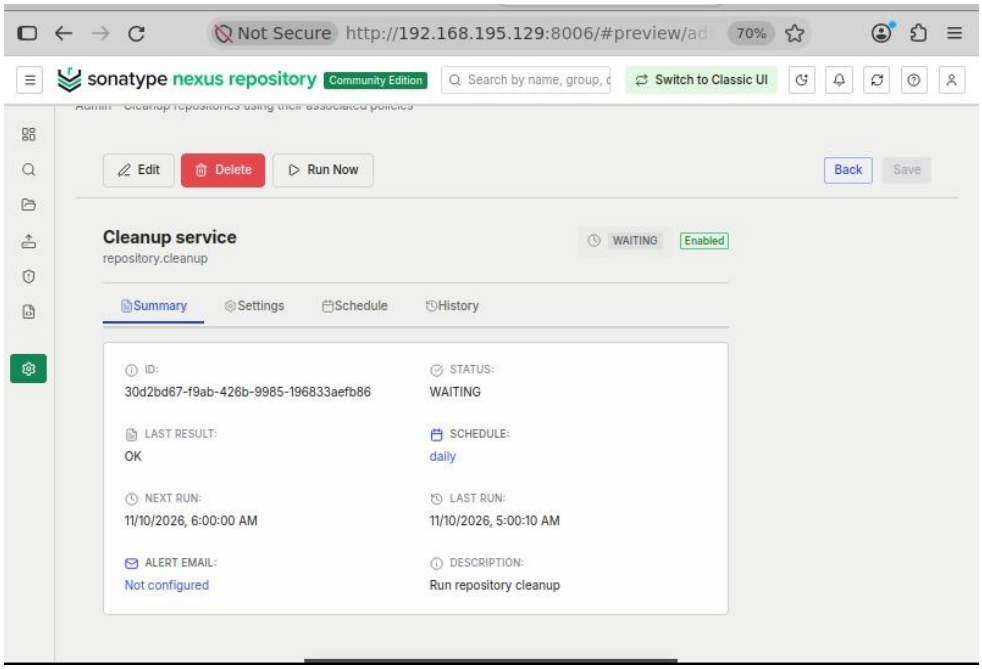
nexus_cleanup_policy_attached.png

The maven policy was moved to the Applied list of the maven-snapshots repository and saved.



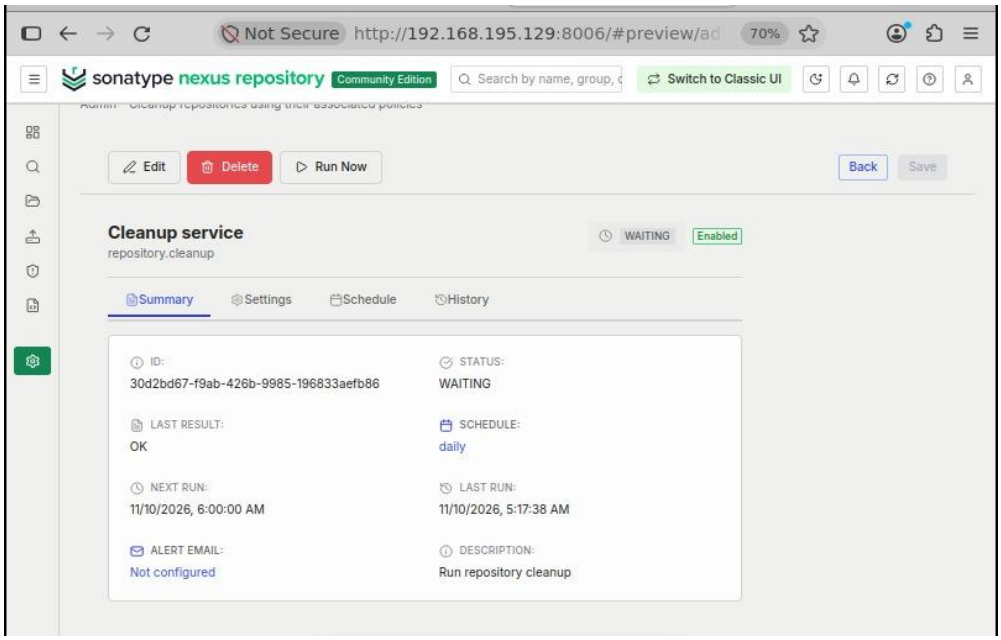
nexus_cleanup_task_waiting.png

The Cleanup service task is in Waiting status and its next run is scheduled for 6:00 AM.



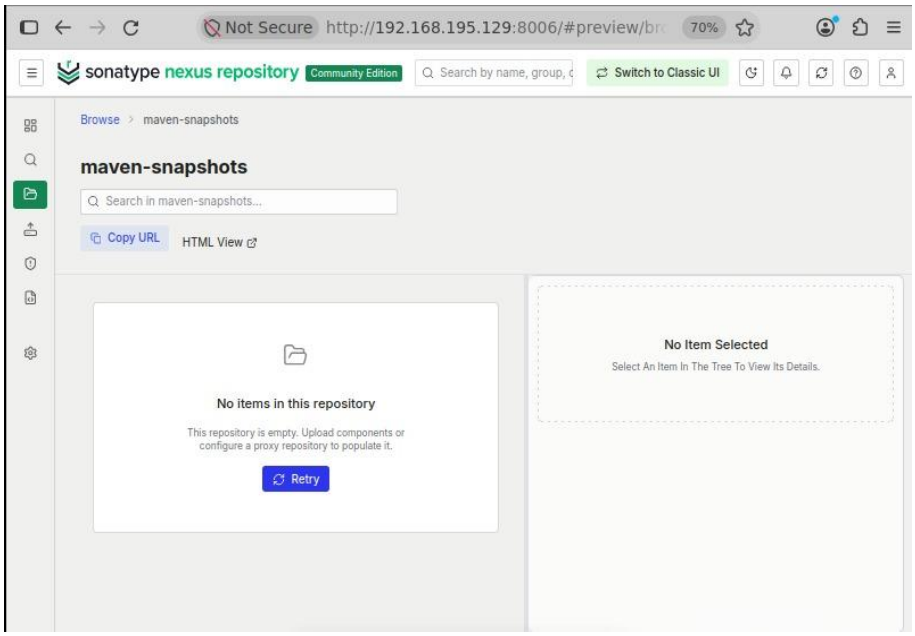
nexus_cleanup_task_completed.png

The Cleanup service was run manually. It returned to Waiting with an updated Last run time and Last result: Ok.



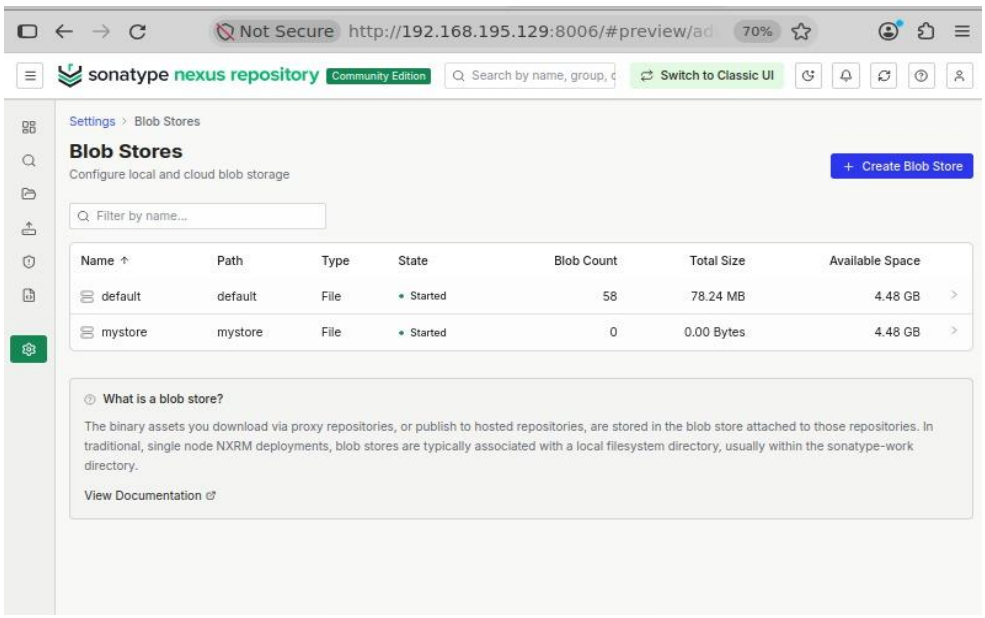
nexus_maven_snapshots_empty.png

After the cleanup service ran, maven-snapshots shows No component/assets found in repository.



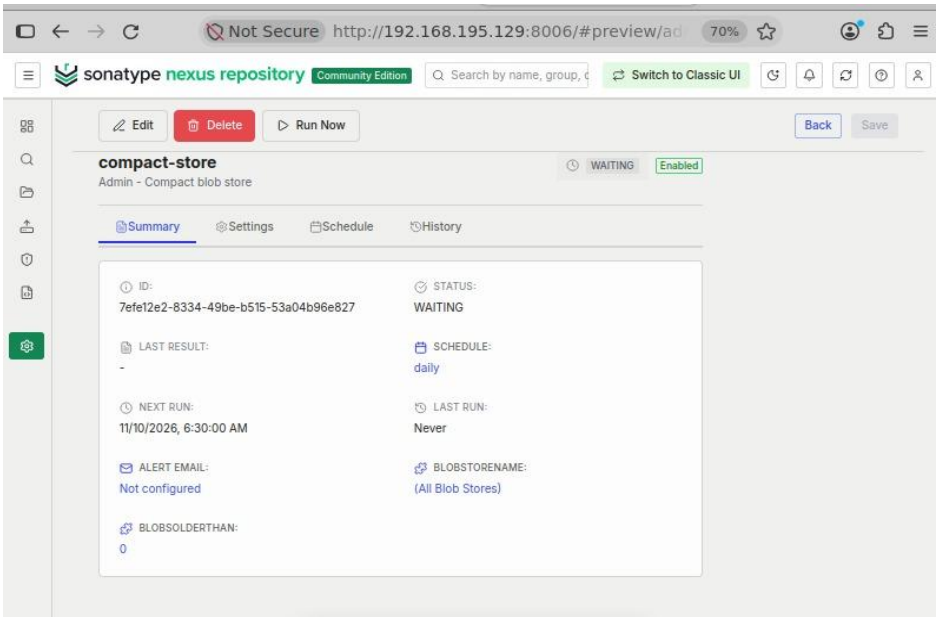
nexus_blob_store_before_compaction.png

Before compaction, the default blob store still shows its blob count, used size and available space because cleanup only soft-deletes components.



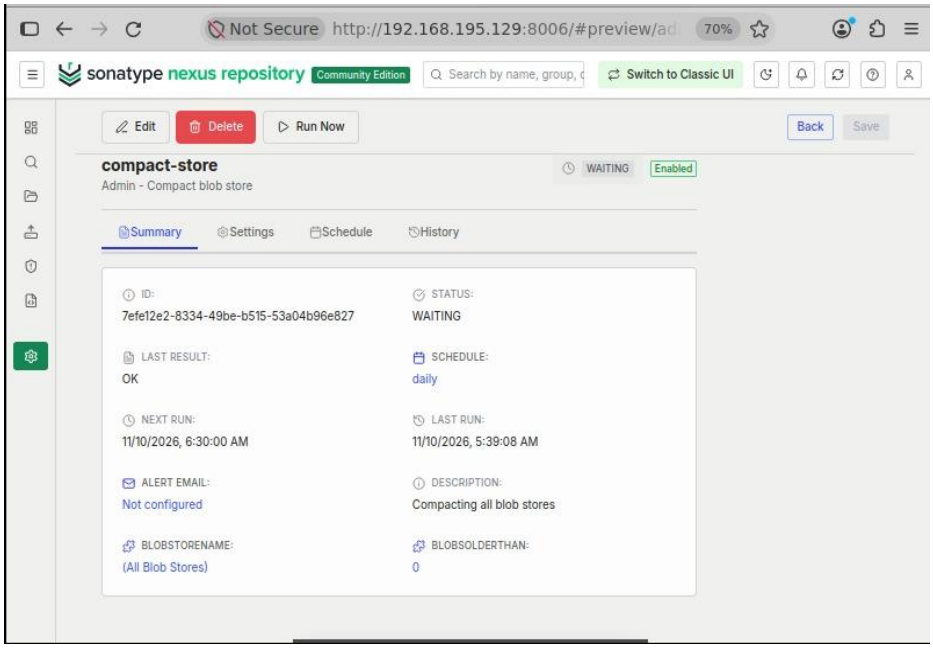
nexus_compact_task_configured.png

The compact-store task (Admin - Compact blob store) was created for the default blob store with Blobs Older Than 0 and a daily schedule at 06:30.



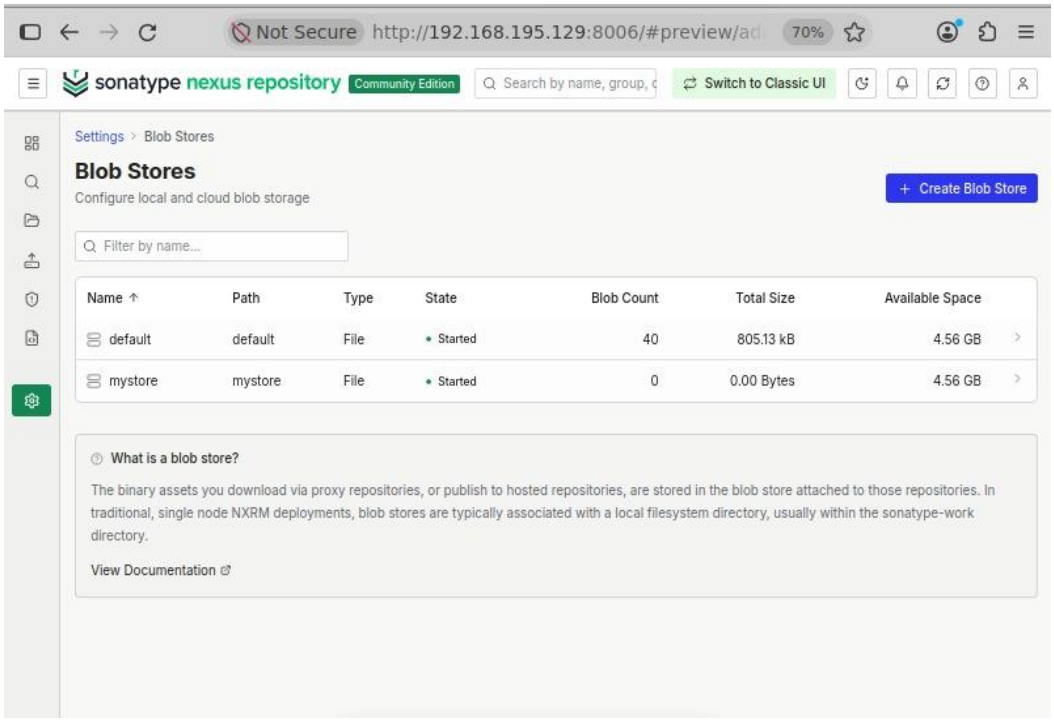
nexus_compact_task_completed.png

The compact-store task was run manually. It returned to Waiting with an updated Last run time and Last result: Ok.



nexus_blob_store_after_compaction.png

After compaction, the default blob store shows a lower blob count and used size and more available space.



Summary: You created a cleanup policy that matches every Maven snapshot component, attached it to maven-snapshots, ran the cleanup service manually, created and ran the compact-store task, and verified that the default blob store reclaimed disk space.